Foundations in Data Engineering

Advanced SQL – part 2

Jana Giceva



Recap from last week

- **Open the TPC-H schema in the Hyper Web-Interface**
- **Q1: How would you compute a `ROLLUP(l_returnflag, l_linestatus)` for the average quantity in the `lineitem` table?**

Today’s agenda

- CTE
- Recursive CTE
- Window functions

Learning Objectives

- **Rewrite** a nested subquery as one or more named `CTE`s
- **Trace** the execution of a recursive `CTE` iteration by iteration, **identifying** the anchor member, the recursive member, the working table, and the termination condition.
- **Write** recursive `CTE`s that generate sequences and **traverse** hierarchies in both directions.
- **Determine** whether a recursive query will terminate on cycling graph data and **justify** the choice between `UNION` and `UNION ALL` and the need for a depth bound.
- **Construct** window-function queries using `PARTITION BY`, `ORDER BY`, and frame clauses to compute running totals, moving averages, rankings, and day-over-day changes with `LAG`/`LEAD`.
- **Predict** the output of a window query, including the effects of the default frame, `ROWS` vs. `RANGE`, and ties under `ROW_NUMBER`, `RANK`, and `DENSE_RANK`.
- **Choose and combine** `GROUP BY`, `CTE`s, and window functions to answer an analytical question in one readable query.

Common Table Expressions
CTEs

Nested queries → CTEs

- Readability and structure
  - Nested queries get hard to read when stacked.
  - What if we could **name each step**, tell the story and keep the outer query simple(r)

```
SELECT dept, avg_grade
FROM (
  SELECT dept, AVG(grade) AS avg_grade
  FROM enrolled
  GROUP BY dept
) t
ORDER BY avg_grade
LIMIT 5;
```

```
WITH t AS (
  SELECT dept, AVG(grade) AS avg_grade
  FROM enrolled
  GROUP BY dept
)
SELECT dept, avg_grade
FROM t
ORDER BY avg_grade
LIMIT 5;
```

- **Q: Does it give us the best or the worst departments?**
- Can reuse the subquery result multiple times.

Common Table Expressions

- Specify a **temporary result** set that can then be referenced by another part of that query.
  - Bind / alias **output columns** to **names** before the AS keyword.

- Alternative to nested queries, views, and explicit temp tables.
- Introduced in the SQL:1999 standard.

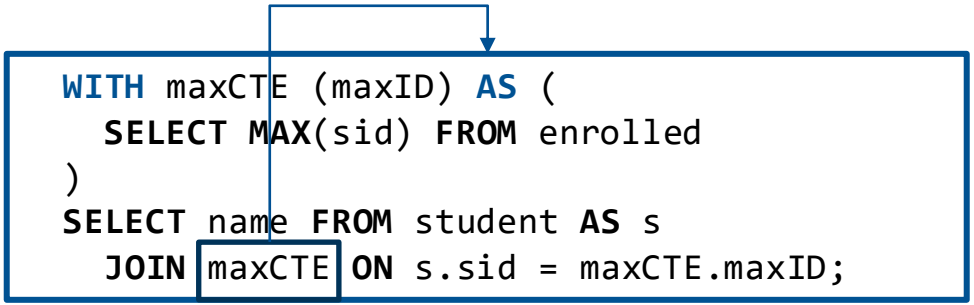
```
WITH cteName (col1, col2) AS (
   SELECT 1, 2
)
SELECT col1 + col2 FROM cteName
```

- `WITH` defines a named temporary result (a ‘view’ for the duration of this query)
- The **outer query** can use that **name** *as if it were a table.*

- Evaluation is non-recursive: it runs the inner query once, that’s it.

Common Table Expressions

- ***Find student record with the highest id that is enrolled in at least one course.***



```
WITH maxCTE (maxID) AS (
  SELECT MAX(sid) FROM enrolled
)
SELECT name FROM student AS s
  JOIN maxCTE ON s.sid = maxCTE.maxID;
```

- ***Q: How would you do this without a CTE?***

```
SELECT name FROM student AS s
  WHERE s.sid = (SELECT MAX(sid) FROM enrolled)
```

Multiple CTE declarations

- A single query may contain multiple CTE declarations:

```
WITH cte1(col1) AS (SELECT 1),
     cte2(col2) AS (SELECT 2)
SELECT * FROM cte1, cte2;
```

- Key point: a later CTE can reference an earlier one, turning a query into a pipeline

```
WITH a AS (…),
     b AS (SELECT … FROM a),
     c AS (SELECT … FROM b)
SELECT … FROM c;
```

Exercise Q4

- Using Uni schema (English) List every professor whose lectures are on average worth more weeklyhours than the average lecture, together with how far above average they are.

Recursive CTEs

Recursive Common Table Expressions

- **What if the data represents a hierarchy or chain?**
  - Like employees and managers, or folders and subfolders,
    or more generally edges in a graph and we want to traverse that relationship.

- ***How would you represent a graph in a relational DBMS?***

- Traversing a graph cannot be done easily with Ordinary CTEs:
  - because we do not know in advance how many levels deep the chain goes.

  → **recursive CTEs.**

- They are called *recursive,* but it is iteration
- Traverse hierarchical data of arbitrary depth (joins only allow a constant number of steps)
- With recursive CTEs, SQL is provably Turing-complete = as expressive as more general purpose PLs.

Recursive Common Table Expressions

- Syntax:

```
WITH RECURSIVE cte_name AS (
  -- 1. anchor: base case
  SELECT ...

  UNION (ALL)

  -- 2. recursive: refers to cte_name itself
  SELECT ...
  FROM cte_name
  JOIN ...
)
SELECT * FROM cte_name
```

The anchor part seeds the recursion (starting rows)

The system automatically unions new results

The recursive part repeatedly extends it (calling itself).

- ***Q: what was the difference between UNION and UNION ALL?***

Recursive Common Table Expressions

- Algorithm:

```
-- first the non-recursive part is executed, producing initial rows
workingTable = evaluateNonRecursive()
result = workingTable

-- then the recursive part is executed using the rows from the working table.
while workingTable is not empty
  workingTable = evaluateRecursive(workingTable)
  result = result U workingTable
output result
```

- When the recursive CTE completes, the union table holds the entire result set, including all intermediate rows from every iteration.

Example

- Before we investigate hierarchies, let's start with simpler numeric recursion

```
WITH RECURSIVE numbers(n) AS (
  -- 1. anchor: base case
  SELECT 1

  UNION ALL

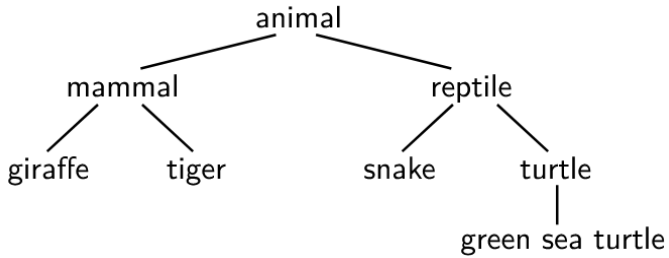
-- 2. recursive: refers to cte_name: numbers
  SELECT n+1
  FROM numbers
  WHERE n < 5
)
SELECT * FROM numbers;
```

- ***Q: What does it output? What if we drop*** `WHERE n < 5`***? What if we write*** `WHERE n <= 5`***?***

Exercises

- ***Q1: Use CTEs to calculate the largest power of 2 that is smaller than 100***
- ***Q2: Compute 10! using recursion***

WITH RECURSIVE … UNION ALL



Finds all ANCESTORS of ‘turtle’

```
WITH RECURSIVE animals (id, name, parent) AS
  (VALUES(1, 'animal', null),
          (2, 'mammal', 1), (3, 'giraffe', 2), (4, 'tiger', 2),
          (5, 'reptile', 1), (6, 'snake', 5), (7, 'turtle', 5),
          (8, 'green sea turtle', 7)),
  r AS (SELECT * FROM animals WHERE name = 'turtle'
        UNION ALL
        SELECT animals.*
        FROM r, animals
        WHERE animals.id = r.parent)
  SELECT * FROM r;
```

Exercise

- ***Q3: Compute all DESCENDANTS of REPTILE.***

Recursive CTEs with `UNION`

- For graph-like data **`UNION ALL`** may not terminate.

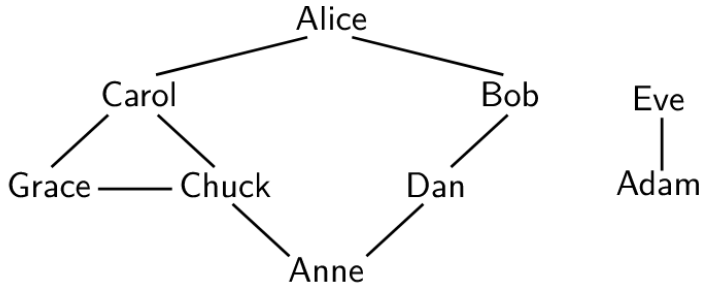
  `WITH RECURSIVE` *[non-recursive]* **`UNION`** *[recursive]*

  – Allows one to traverse cyclic structures

- **Algorithm:**

```
workingTable = unique(evaluateNonRecursive())
result = workingTable
while workingTable is not empty
  workingTable = unique(evaluateRecursive(workingTable)) \ result
  result = result U workingTable
output result
```

WITH RECURSIVE … UNION




```
WITH RECURSIVE
  friends(a,b) AS (VALUES ('Alice', 'Bob'), ('Alice', 'Carol'), ('Carol', 'Grace'),
    ('Carol', 'Chuck'), ('Chuck', 'Grace'), ('Chuck', 'Anne'), ('Bob', 'Dan'),
    ('Dan', 'Anne'), ('Eve', 'Adam')),
  friendship(name, friend) AS (SELECT a, b FROM friends UNION ALL SELECT b, a FROM friends),
  r(name) AS (
        SELECT 'Alice'
        UNION
        SELECT f.name FROM r
        JOIN friendship f ON f.friend = r.name)
  SELECT * FROM r ORDER BY name;
```

Exercise

- *Q: Show the levels by distance from Alice. Be wary of infinite loop.*

Window functions

From CTEs to Window functions

- CTEs are a great way to query over sets of data that depend on other sets.
  - Structure *complex query logic* – by building derived tables and dependencies between sets of data.

- Window functions extend this idea – but at a row level, not set level.
  - Structure *analytical logic* – allowing us to perform complex calculations across rows, without collapsing them into groups.
  - *e.g.*, compute running totals, rankings, and moving averages *without* losing the individual rows.

- ***Example:***
  What happens if we use `GROUP BY` to find total sales per customer?

```
SELECT customer_id, sale_date, amount,
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY sale_date) AS running_total
FROM sales;
```

What if we still want to see the running sum of sales per customer?

Window Functions

- Part of SQL since SQL:2003
- Uses multiple rows to calculate a value for each row – like aggregations that do not collapse rows.
- Very versatile feature:
  - Time series analysis, ranking, top-k, percentiles, running totals, moving averages, percentiles
- Window functions are evaluated after most other clauses (including **`GROUP BY`**) but before **`ORDER BY`**.

```
function_name(expression) OVER (
  PARTITION BY . . .
  ORDER BY . . .
  ROWS BETWEEN . . .
)
```

`OVER()` keyword defines the window, i.e, which rows are visible.

Window Functions: key concepts

- **Calculate across a set of tuples, related to the current tuple**, without collapsing them to support running totals, ranks, and moving averages.
  - Like an aggregation, but tuples are not grouped into a single output tuple.

Aggregation or special **functions**, computed over the window frame

`OVER()` keyword define the window, i.e, which rows are visible.

```
func_name(...) OVER (
    PARTITION BY ...
    ORDER BY ...
    ROWS | RANGE BETWEEN <start> AND <end>
)
```

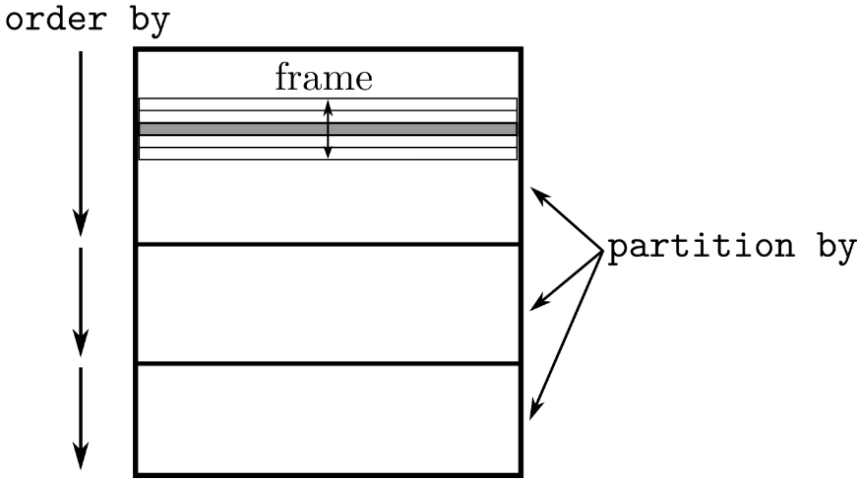
partition clause – which *group of rows* you're working in (mini dataset)

order clause – in what order they are in

frame clause – *how far the function's reach extends* within that ordered partition

Window Functions: frames

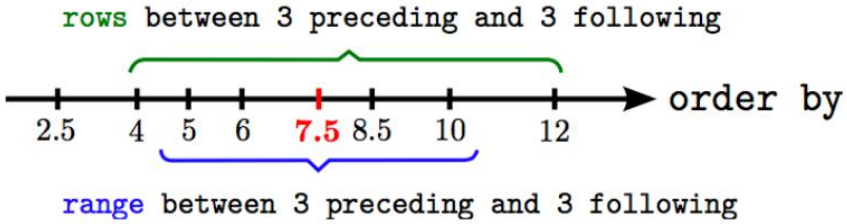
- *Frame* is like a *moving slice of the partition* that slides along as you move from one row to the next.



- When we omit to explicitly write the frame – we fallback to using the default frame.
  - With `ORDER BY`, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`
  - If no `ORDER BY`, the entire partition is used (i.e., all rows)

Frame clauses

- **Frame units**
  - `ROWS` – count-based
    (e.g., 2 rows before)
  - `RANGE` – value-based
    (e.g., all rows within 10 units of this value)



- **Frame boundaries**
  - Each frame has **start** and **end** boundaries
  - `UNBOUNDED PRECEDING` – start from the first row in the partition
  - `n PRECEDING` – start n rows (values) before the current one
  - `CURRENT ROW` – start or end at this row
  - `n FOLLOWING` – extend n rows (values) after the current one
  - `UNBOUNDED FOLLOWING` – go all the way to the end

Example

- Running total

```
-- everything from start to current row
SELECT date,
       amount,
       SUM(amount) OVER (
       ORDER BY date
       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM sales;
```

Date	Amount
2025-01-01	10
2025-01-02	20
2025-01-03	15
2025-01-04	30

date	amount	running_total
2025-01-01	10	10
2025-01-02	20	30
2025-01-03	15	45
2025-01-04	30	75

Example

- 3 day moving average

```
-- 2 preceding and current row
SELECT date,
       amount,
       AVG(amount) OVER (
       ORDER BY date
       ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_avg
FROM sales;
```

Date	Amount
2025-01-01	10
2025-01-02	20
2025-01-03	15
2025-01-04	30

date	amount	moving_avg
2025-01-01	10	10
2025-01-02	20	15
2025-01-03	15	15
2025-01-04	30	21.67

The frame has a *fixed size* that slides along the ordered rows

Example window functions

- **Two main families of window functions**
  - **Aggregate** (frame-sensitive) window functions:
    - *e.g.,* `SUM, AVG, COUNT, MIN, MAX, VARIANCE, STDDEV, etc.`
    - Compute results over a set(frame) of rows and are affected by the frame clause.
  - **Value** (frame-insensitive) window functions
    - Functions *pick or compare specific rows* within the partition – they do not aggregate over a frame
    - They use *ordering* but *ignore frames.*
    - *e.g.,* `LAG, LEAD, RANK(), DENSE_RANK(), ROW_NUMBER(), etc.`

rank allows for gaps:
RANK(gaps): 1,1,3,4,4
DENSE RANK: 1,1,2,3,3

  - Rows that are not distinct when considering only the ORDER BY columns are said to be *peers.*

Value-based window functions

- `LAG(expr, offset, default)` – value from previous row (e.g., looks backward)
- `LEAD(expr, offset, default)` – value from a following row (e.g., looks forward)
- `FIRST_VALUE(expr)` – value from the first row in the partition (by `ORDER BY`)
- `LAST_VALUE(expr)` – value from the last row in the partition (by `ORDER BY`)
- `NTH_VALUE(expr, n)` – value from the n-th row in the partition
- `RANK()` – assigns ranks with gaps for ties
- `DENSE_RANK()` -- assigns ranks without gaps for ties
- `ROW_NUMBER()` – sequential numbering
- `NTILE(n)` – divides rows into n buckets
- `CUME_DIST()` – relative position (cumulative distribution)
- `PERCENT_RANK()` – relative rank of the current row ((rank-1)/(partition rows -1))

Exercise

- **Context:**
  - Online retail dataset that records daily sales for different regions.
  - We want to help business analysts answer questions like:
    - *How are sales trending in each region over time?*
    - *Which days record highs or lows?*
    - *How volatile are recent sales?*

- **Task: write SQL query that shows for each region and day**
  1. The total sales on that day
  2. The running total of sales up to that day
  3. The 3-day moving average of sales
  4. The rank of that day's sales **within the region**

  How can we compute all these while keeping each day visible?

Region	Sale date	Sales
East	2025-01-01	1000
East	2025-01-02	1200
East	2025-01-03	900
East	2025-01-04	1300
East	2025-01-05	1500
West	2025-01-01	800
West	2025-01-02	950
West	2025-01-03	1100
West	2025-01-04	900
West	2025-01-05	1000

Exercises

- **Q: Could we compute sales difference vs. previous day?**
  - Yes, we can compute the difference with `LAG(sales) OVER ()` and a simple subtraction

```sql
SELECT region, sale_date,
-- difference to previous day
sales - LAG(sales) OVER (
  PARTITION BY region ORDER BY sale_date
) AS day_over_day_change FROM sales_data;
```

Q: How can we handle the `NULL` on day 1?

Q: How about a percentage change?

Tell the system to interpret NULL as zero

- **Q: Given the following simple dataset: predict the output of this query.**

date	sales
2025-01-01	10
2025-01-02	20
2025-01-03	15
2025-01-04	30

```sql
SELECT date, sales,
       SUM(sales) OVER (ORDER BY date) AS run_sum,
       LAG(sales) OVER (ORDER BY date) AS prev_sale,
       RANK() OVER (ORDER BY sales DESC) AS rank_desc
FROM t;
```

This might be a kind of question for the exam

WINDOW clauses

- If we want to use the same layout for multiple window functions, the `WINDOW` clause can be used to define a *named* window that can be shared (and reused) between multiple window functions.

```
SELECT plant, date,
   min("MWh") OVER seven AS "MWh 7-day Moving Minimum",
   avg("MWh") OVER seven AS "MWh 7-day Moving Average",
   max("MWh") OVER seven AS "MWh 7-day Moving Maximum"
FROM generation_history
WINDOW seven AS (
   PARTITION BY plant
   ORDER BY date ASC
   ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING
)
ORDER BY 1, 2;
```

Advanced SQL takeaways

- Aggregate: summarize data at several levels at once
  - GROUPING SETS, ROLLUP and CUBE replace stacks of UNION ALL queries.
  - GROUPING() tells subtotal NULLs apart from real NULLs.

- Structure: name the steps of a complex query
  - CTEs (WITH) break a query into readable, reusable building blocks.
  - Recursive CTEs (WITH RECURSIVE) iterate until no new rows appear. They traverse hierarchies and graphs of unknown depth.
  - Use UNION ALL when the data is acyclic. Use UNION plus a bound when there may be cycles.

- Analyze: compute across rows without collapsing them
  - Window functions follow the pattern func() OVER (PARTITION BY … ORDER BY … frame).
  - Running totals, moving averages, rankings and LAG/LEAD all work while keeping every row.
  - Watch out for the default frame, ROWS vs. RANGE and ties.

Modern SQL

- Is more than SELECT-FROM-WHERE:
  - One can push the computation to the database instead of exporting the data and looping in Python.

- Today we stored a graph as an edge table and needed recursive CTEs to find Alice's friends of friends. Is the relational model still the right fit?

  - Next time: data models.

References

- Basic SQL (in German):
  - Datenbanksysteme: Eine Einführung, Alfons Kemper and Andre Eickler, 10th edition, 2015
- Joe Celko's SQL for Smarties, Joe Celko, 5th edition, 2014
- SQL cookbook, Anthony Molinaro, 2005

- Lecture:
  - Foundations in Data Engineering – Advanced SQL by Prof. Neumann (TUM)
  - Database Systems – Modern SQL by Prof. Pavlo (CMU)

- Web materials:
  - DuckDB and PostgreSQL SQL documentation

- Testing environment:
  - https://hyper-db.de/interface.html#